Question 01

A library wants to manage the lending of books to members. Write a Java program to handle book lending and update the availability of books accordingly.

a) Write a class called **LibraryBook** to represent the details of the books in the library with the following private attributes:

- bookId
- title
- author
- copiesAvailable

b) Implement a constructor that accepts bookId, title, author, and copiesAvailable as parameters.

c) Implement a method called lendBook(), which accepts the number of copies requested by a member and displays appropriate messages. It should:

- If copiesAvailable is less than the requested number of copies, display: "Not enough copies available. Please check back later."
- Otherwise:
  - Deduct the requested quantity from the available copies.
  - Display: "Book issued successfully!"

d) Implement a method called printBookDetails() to display the complete details of the book, including how many copies are still available.

e) Write another Java class called LibraryApp to perform the following tasks:

I. Input details of two books from the user and create two LibraryBook objects.

II. Input the number of copies each member wants to borrow for each book, and process the lending using the lendBook() method.

III. After lending operations, display the details of all the books using the printBookDetails() method.

Question 02

A Transport Management System manages different types of vehicles used in a transportation company. Each vehicle has a brand and speed, but there are specialized vehicles such as Buses and Trucks that have unique features.

Each Vehicle contains an Engine that determines its power, meaning Engine is a part of the Vehicle and cannot exist without it.

Additionally, the transportation company owns multiple Drivers, and each Driver is associated with a Vehicle. However, a Driver can exist independently of a specific vehicle.

Different types of vehicles have unique implementations of the start() method, which need to be overridden in the respective classes

a) Implement a class called **Vehicle** with brand(String) and speed(double) attributes.
    i. Create a method called start(), that prints a generic message.
b) Create two classes called **Bus** and **Truck** as subclasses of Vehicle.
    i. Override the start() method to display unique messages for each type.
c) Implement an **Engine** class with engineType(String) as an attribute.
    i. The Engine should be part of a Vehicle (Composition) and initialized within the Vehicle constructor.
d) Create a class called **Driver** with name(String) and licenseNumber(String) attributes.
    i. A Driver should be associated with a Vehicle using Aggregation
e) Modify the **Bus** and **Truck** classes to include a Driver for each class(Aggregation).
f) Create a **TransportApp** class with a main method to:
    i. Create objects for different Vehicles (Bus, Truck) with their Engines.
    ii. Assign Drivers to those Vehicles.
    iii. Start the vehicles and display full details, including the associated Driver and Engine.

Sample Output:

```
--- Bus Details ---
Bus is starting... Passengers are boarding.

Vehicle Brand: Volvo
Speed: 80.0 km/h
Engine Type: Diesel
Driver Name: John Doe, License: DL12345
Passenger Capacity: 50

--- Truck Details ---
Truck is starting... Loading goods.

Vehicle Brand: Mercedes
Speed: 60.0 km/h
Engine Type: Turbo Diesel
Driver Name: Alice Smith, License: DL67890
Load Capacity: 10.0 tons
```

Question 03

Write a program that prompts the user to enter a username and password, then perform the following validations using two different custom exception classes as follows.

- **InvalidUserName**, is an exception class that prints out the error message "*The username must be at least 6 characters long*" if the number of characters in the user input for the username is less than 6.
- **InvalidPasswordLength**, is an exception class that prints out the error message "*The password must be at least 8 characters long"* if the number of characters in the user input for the password is less than 8.

Write another class called **DemoApp** to get the user inputs for the username, and password and then validate those for the above two conditions. If the input has one or more custom exceptions, you should have proper try-catch statements to handle the exceptions.

Sample Output:

```
Enter a username: Kamal
Enter a password: Kamal123
Error: Username must be at least 6 characters long.
```

```
Enter a username: KamalPerera
Enter a password: Ka1
Error: Password must be at least 8 characters long.
```

Question 04

Develop a Java program to manage a **Library Inventory** using an ArrayList. The program should perform the following tasks:

a) Define a **Book** class with two attributes as isbn, and title. Include a constructor to initialize both fields and a method *displayDetails()* that prints the book's ISBN and title.

b) Implement a class, create an empty ArrayList named inventory. Prompt the user to enter book details one at a time: first the ISBN, then the title. Keep accepting entries until the user enters "-99" as the ISBN. For each valid entry, instantiate a new Book and add it to inventory.

c) After input is complete, prompt the user to enter the ISBN of the book they wish to remove from the inventory.

- If a Book with that ISBN exists in inventory, remove it and display "***Book removed successfully***.".
- If not found, display "***ISBN not found. No book removed***."

d) Display the total number of books currently in inventory.

e) Print all remaining books in the inventory by calling each object's *displayDetails()* method.